

Python Basics, Programming Fundamentals

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 01, Lecture

19 September 2026

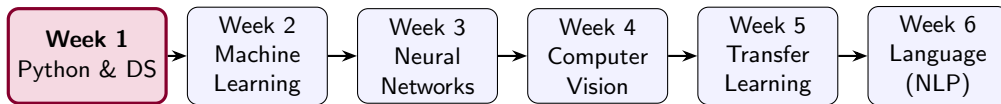
1. Big Picture and Motivation
2. Learning Outcomes
3. Your First Program
4. Variables and Types
5. Numbers and Strings
6. Booleans and Decisions
7. Lists
8. Loops
9. Dictionaries
10. Functions
11. Reading Errors
12. Classes and Objects
13. Importing Modules

14. Olympiad Practice

15. Limitations and What Comes Next

16. Summary

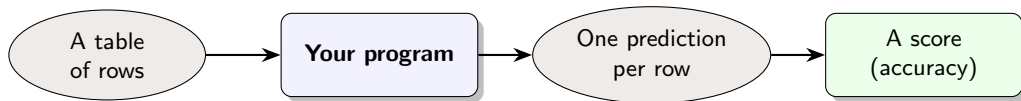
17. References



Today: Day 1 of 42

- ▶ **Today:** we learn to give a computer instructions in Python.
- ▶ **This week:** Python (Sat), the math of vectors and matrices (Sun), tables and plots (Mon).
- ▶ **Why now:** every later week is Python code built on top of today's ideas.

What an AI Olympiad Task Looks Like



- ▶ You get data and a question ("which flower is this row?").
- ▶ You write a program that answers the question for every row.
- ▶ You hand in a file of answers. The organisers count how many are right.

The file you hand in

```
id,label  
1,setosa  
2,versicolor  
3,setosa
```

One line per row. This is the Kaggle format we use every Friday.

A class quiz. Five students, one score each. Find the average and the top score.

Student	Score
Ali	78
Sara	92
Omar	85
Lina	64
Yara	91

- ▶ **Add:** $78 + 92 + 85 + 64 + 91 = 410$
- ▶ **Divide:** $410/5 = 82$
- ▶ **Top score:** read down the column, keep the largest: 92 (Sara)

Five rows: about one minute, and you can check it by eye.

The same job on a real dataset

The UCI Adult dataset (a classic Olympiad-style table) has **32,561 rows** and 14 columns. At 3 seconds per row, adding one column by hand takes $32,561 \times 3 \approx 97,700$ seconds, about **27 hours**. One slip anywhere, and the average is wrong and nobody notices.

- ▶ **Scale:** thousands of rows, dozens of columns, many questions per table.
- ▶ **Errors:** humans lose their place; the mistake hides inside the total.
- ▶ **Repetition:** the task is the same for every row. Only the numbers change.

In one sentence

Write the steps **once**, and let Python repeat them on every row, in milliseconds, exactly the same way each time.

Where you will meet it

Every Olympiad notebook is Python: load the data, loop over the rows, compute a score, write the answer file. Today you learn the pieces: values, lists, loops, functions and a first look at objects.

By the end of this session, you will be able to:

- ▶ **Explain** what a program and a variable are, and run a cell in a notebook.
- ▶ **Use** numbers, strings, booleans, lists and dictionaries to store data.
- ▶ **Write** decisions and loops with `if`, `for` and `while`.
- ▶ **Write** a function, call it, and say what goes in and what comes out.
- ▶ **Explain** the difference between a class and an object, and build a small one.
- ▶ **Read** an error message and fix the line it points to.

Python Basics: **Your First Program**

Definition

A **program** is a list of instructions written in a language the computer understands. Python reads the lines from top to bottom and carries out each one.



- ▶ **Input → output:** text you wrote goes in, results come out.
- ▶ **Not to be confused with:** a calculator does one step at a time. A program does many steps and remembers results in between.

We write Python in **Google Colab**, a notebook in the browser. A notebook is a list of **cells**.

```
[1]: print("Hello")
```

Hello

A text cell: notes for yourself

```
[2]: print(2 + 3)
```

5

- ▶ **Code cell:** you type Python, press Shift+Enter, the output appears below.
- ▶ **Text cell:** your notes and headings.
- ▶ **Order matters:** cells run in the order *you* run them. “Restart and run all” starts from the top.

```
print("Hello , Olympiad!")    # Hello , Olympiad!  
print(2 + 3)                  # 5  
print("2 + 3")                # 2 + 3
```

- ▶ `print(...)` shows a value below the cell.
- ▶ Text inside quotes is shown *as it is*. Without quotes, Python *calculates* it.
- ▶ Everything after `#` is a **comment**: Python ignores it. It is for humans.

```
print(hello)
```

```
Traceback (most recent call last):  
  File "<cell>", line 1, in <module>  
NameError: name 'hello' is not defined
```

Read the last line first

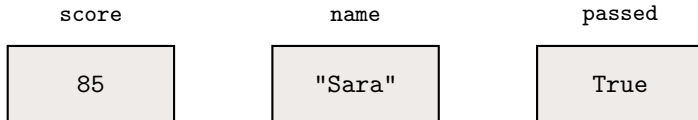
The last line says **what** went wrong (NameError: Python does not know a name) and the line above says **where** (line 1). We wanted text, so we forgot the quotes.

Errors are normal. Every programmer reads dozens per day. The skill is reading them calmly.

Python Basics: **Variables and Types**

Definition

A **variable** is a name that points to a value. `score = 85` puts the value 85 into a box called `score`.



- ▶ The **name** is on the left, the **value** on the right.
- ▶ `=` means “store this value under this name”. It does not mean “equals”.
- ▶ Names: letters, digits and `_`, no spaces, cannot start with a digit. `round1` is fine, `1round` is not.

```
score = 85  
score = score + 5  
print(score)          # 90
```

The right side runs first

`score = score + 5` is not an equation. Python first computes the right side ($85 + 5$), then stores the result in the box.

Line	What Python does	Box score
1	store 85	85
2	read 85, add 5, store 90	90
3	read 90, show it	90

Every value has a **type**. The type decides what you can do with the value.

Type	Name	Example	What it holds
int	integer	42	whole numbers
float	float	3.14	numbers with a decimal point
str	string	"Sara"	text, always in quotes
bool	boolean	True	yes or no (True or False)

```
type(42)           # <class 'int'>
type("42")        # <class 'str'>    quotes make it text
type(4 / 2)        # <class 'float'> division always gives a float: 2.0
```

<code>int("42")</code>	<code># 42</code>	text to whole number
<code>float("3.5")</code>	<code># 3.5</code>	text to decimal number
<code>str(7)</code>	<code># "7"</code>	number to text
<code>int(3.7)</code>	<code># 3</code>	cuts off the decimals, does not round

Where it breaks

`int("3.7")` gives `ValueError: the text is not a whole number`. Use `float("3.7")`.

`"42" + 1` gives `TypeError: you cannot add text and a number`. Convert first.

Data files store everything as text. The first thing a program does is convert.

```
line = "85"           # what we read from a file: text
score = int(line)      # now a number: 85
bonus = score + 10     # 95
print(type(line), type(score))
# <class 'str'> <class 'int'>
```

- ▶ Line 1: the box `line` holds the text "85" (two characters).
- ▶ Line 2: `int` reads the text and makes the number 85.
- ▶ Line 3 works only because `score` is a number. In Lab 2 you do this for 150 rows.

Python Basics: **Numbers and Strings**

Operator	Meaning	Example	Result
+	add	7 + 2	9
-	subtract	7 - 2	5
*	multiply	7 * 2	14
/	divide	7 / 2	3.5 (always a float)
//	whole part	7 // 2	3
%	remainder	7 % 2	1
**	power	2 ** 10	1024

- ▶ Order is the school order: powers, then * / // %, then + -. Brackets first.
- ▶ % answers “is it even?”: 8 % 2 is 0, 9 % 2 is 1.

$$\bar{x} = \frac{x_1 + x_2 + x_3}{3}$$

► x_1, x_2, x_3 : the three scores $\bar{x}$ ("x bar"): their average

► **Worked example:** $x_1 = 85, x_2 = 92, x_3 = 78$, so $\bar{x} = \frac{85 + 92 + 78}{3} = \frac{255}{3} = 85.0$

```
average = (85 + 92 + 78) / 3  
print(average)                # 85.0
```

Without the brackets Python would compute $85 + 92 + 78/3 = 203.0$. Brackets first.

A **string** is a row of characters. Each character has a position, counted from 0.

index	0	1	2	3	4	5
	P	Y	T	H	O	N
from the end	-6	-5	-4	-3	-2	-1

- ▶ `s = "PYTHON"` `len(s)` is 6 `s[0]` is "P" `s[-1]` is "N"
- ▶ `s[0:3]` is "PYT": start at 0, stop *before* 3.
- ▶ Single or double quotes both work: 'Sara' and "Sara" are the same.

A **method** is an action a value knows how to do. You call it with a dot.

```
name = "  Sara Ali "  
name.strip()           # "Sara Ali"           remove spaces at both ends  
name.strip().lower()   # "sara ali"           chain two methods  
name.strip().upper()   # "SARA ALI"  
name.split()           # ["Sara", "Ali"]       cut into a list of words  
"cat".replace("c", "b") # "bat"  
"Sara" + " " + "Ali"   # "Sara Ali"           + glues strings
```

Methods do not change the original: `name` is still `" Sara Ali "` afterwards. Store the result if you need it.

```
name = "Sara"  
score = 85.6667  
print(f"{name} scored {score}")           # Sara scored 85.6667  
print(f"{name} scored {score:.1f}")       # Sara scored 85.7
```

- ▶ The `f` before the quote turns on the braces: `{name}` is replaced by the value.
- ▶ `:.1f` means “show 1 digit after the decimal point”. `:.2f` shows two.
- ▶ This is how you print results in every lab and how you write submission lines on Friday.

Python Basics: **Booleans and Decisions**

Operator	Meaning	Example	Result
<code>==</code>	equal	<code>5 == 3</code>	False
<code>!=</code>	not equal	<code>5 != 3</code>	True
<code><</code>	less than	<code>5 < 3</code>	False
<code>>=</code>	at least	<code>5 >= 5</code>	True
<code>==</code>	equal (text)	<code>"a" == "A"</code>	False (case matters)

One or two equal signs

`=` stores a value. `==` asks a question. `score = 90` changes the box; `score == 90` answers True or False.

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

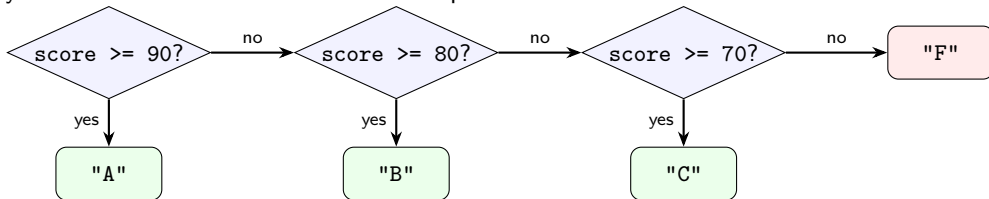
`not True` is `False`.

Worked example. Is a score valid?

- ▶ `score = 85`
- ▶ `score >= 0` is `True`
- ▶ `score <= 100` is `True`
- ▶ `True and True` is `True`: the score is valid.

With `score = 120` the second part is `False`, so the whole `and` is `False`.

Python checks the conditions from the top and runs the **first** block whose condition is True.



With score = 85: first question no, second question yes, grade "B". The third question is never asked.

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(f"Grade: {grade}")      # Grade: B
```

if once, elif ("else if") as many times as you need, else at most once, at the end.

The four spaces after the colon tell Python which lines belong to the if.

Correct

```
if score >= 50:
    print("passed")
    print("well done")
print("done")    # always runs
```

Error

```
if score >= 50:
    print("passed")
# IndentationError:
# expected an indented block
```

The most common beginner mistake

A line indented by accident joins the block above it. A line not indented leaves the block. Colab indents for you after every colon; keep it.

Python Basics: **Lists**

Definition

A **list** holds many values in order. Each value has a position (its **index**), counted from 0.

```
scores = [78, 92, 85, 64, 91]
```

index	0	1	2	3	4
	78	92	85	64	91
from the end	-5	-4	-3	-2	-1

- ▶ The first item is at index 0, the last at index `len(scores) - 1`, which is 4 here.
- ▶ A list can hold anything: numbers, strings, even other lists.

```
scores = [78, 92, 85, 64, 91]
scores[0]          # 78          first
scores[-1]         # 91         last
scores[1:3]        # [92, 85]   from index 1, stop before 3
scores[:2]         # [78, 92]   the first two
scores[3:]         # [64, 91]   from index 3 to the end
```

- ▶ A **slice** `[a:b]` takes indices $a, a + 1, \dots, b - 1$. It has $b - a$ items.
- ▶ `scores[5]` is an `IndexError`: there is no index 5 in a list of 5 items.

```
scores = [78, 92, 85]
scores.append(64)      # [78, 92, 85, 64]    add at the end
scores[0] = 80         # [80, 92, 85, 64]    replace one item
scores.pop()          # returns 64, list is [80, 92, 85]
len(scores)           # 3
sum(scores)            # 257
min(scores), max(scores) # 80, 92
sorted(scores)         # [80, 85, 92]    a new sorted list
```

append changes the list in place. sorted gives a new list and leaves the old one alone.

```
table = [[78, 92, 85],  
         [64, 91, 70],  
         [88, 79, 95]]
```

	col 0	col 1	col 2
row 0	78	92	85
row 1	64	91	70
row 2	88	79	95

- ▶ Each inner list is one **row**: one student, three rounds.
- ▶ `table[1]` is row 1: `[64, 91, 70]`.
- ▶ `table[1][2]` is row 1, column 2: 70.
- ▶ `len(table)` is the number of rows: 3.

This is the shape of every dataset

Rows are examples, columns are measurements. Tomorrow NumPy and on Monday pandas give this table a faster home.

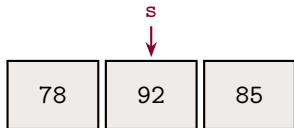
```
shape = (28, 28)           # round brackets make a tuple
shape[0]                   # 28 indexing works like a list
shape[0] = 32              # TypeError: cannot change a tuple
```

- ▶ Use a tuple for things that should stay fixed: an image size, a pair of coordinates.
- ▶ You will mostly *read* tuples, not build them: tomorrow `array.shape` returns one, such as `(150, 4)`: 150 rows, 4 columns.

Python Basics: **Loops**

for: Do This for Every Item

```
scores = [78, 92, 85]
for s in scores:
    print(s * 2)
```



s visits each box in turn

Step	s	Printed
1	78	156
2	92	184
3	85	170

- ▶ s is a variable that takes each value in turn.
- ▶ The indented block runs once per item, then the loop ends.

```
for i in range(5):           # 0 1 2 3 4   stops before 5
    print(i)
for i in range(1, 6):       # 1 2 3 4 5   (start, stop)
    print(i)
for i in range(0, 10, 2):   # 0 2 4 6 8   (start, stop, step)
    print(i)
```

- ▶ `range(n)` gives the n numbers $0, 1, \dots, n - 1$, exactly the indices of a list of length n .
- ▶ So for `i in range(len(scores))` visits every index, and `scores[i]` is the item there.

To add up a list, keep a running total and add each item to it.

$$\text{total} = \sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

x_i : the i -th score n : how many scores $\sum$: “add them all up”

Step	Item x_i	Total after
start	–	0
1	78	78
2	92	170
3	85	255
4	64	319
5	91	410

- ▶ The total starts at 0 *before* the loop.
- ▶ Each step: new total = old total + item.
- ▶ After the loop: 410, and the average is $410/5 = 82$.

```
scores = [78, 92, 85, 64, 91]
total = 0                                # start before the loop
for s in scores:
    total = total + s                    # or: total += s
average = total / len(scores)
print(total, average)                   # 410 82.0
```

- ▶ Same pattern for counting: start count = 0, add 1 inside an if.
- ▶ Same pattern for the largest: start best = scores[0], replace it when you see a bigger one.
- ▶ Python's `sum(scores)` does the same thing. You write it by hand today so you know what it does.

while: Repeat Until a Condition Fails

```
loss = 8.0
steps = 0
while loss > 1:
    loss = loss / 2
    steps = steps + 1
print(steps, loss)    # 3 1.0
```

Check loss > 1	loss after	steps
8.0 > 1: yes	4.0	1
4.0 > 1: yes	2.0	2
2.0 > 1: yes	1.0	3
1.0 > 1: no	stop	

- ▶ Use `while` when you do not know in advance how many times to repeat.
- ▶ `break` leaves any loop at once. If the condition never becomes `False`, the loop runs forever: stop the cell with the square button.
- ▶ Training a model in Week 3 is a loop like this: repeat until the loss is small enough.

zip and enumerate: Two Lists Side by Side

```
y_true = ["cat", "dog", "cat"]
y_pred = ["cat", "dog", "dog"]
for t, p in zip(y_true, y_pred):      # both lists together
    print(t, p, t == p)              # cat cat True ... cat dog False

for i, p in enumerate(y_pred):        # index and item together
    print(i, p)                       # 0 cat / 1 dog / 2 dog
```

- ▶ zip pairs the first with the first, the second with the second, and so on.
- ▶ enumerate numbers the items from 0. It gives you the id column of a submission file.

The loop

```
squares = []  
for x in range(5):  
    squares.append(x ** 2)  
# [0, 1, 4, 9, 16]
```

The same in one line

```
squares = [x ** 2 for x in range(5)]  
# [0, 1, 4, 9, 16]  
  
passed = [s for s in scores if s >= 70]
```

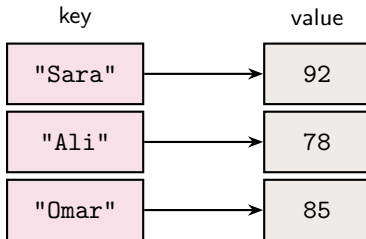
Read it as “*the list of $x ** 2$ for every x in $\text{range}(5)$* ”. The optional `if` keeps only some items. Use it when the loop body is one expression; otherwise write the loop.

Python Basics: **Dictionaries**

Definition

A **dictionary** stores pairs: a **key** and its **value**. You look up a value by its key, not by a position.

```
scores = {"Sara": 92, "Ali": 78, "Omar": 85}
```



- ▶ Like a real dictionary: you look up a word (key) and get its meaning (value).
- ▶ Keys are usually strings; values can be anything. Each key appears once.

```
scores = {"Sara": 92, "Ali": 78}
scores["Sara"]           # 92                read by key
scores["Omar"] = 85      # add a new pair
scores["Ali"] = 80       # update an existing value
"Omar" in scores         # True              is the key there?
scores["Lina"]           # KeyError: 'Lina' is not a key
len(scores)              # 3
```

- ▶ Reading a missing key is an error. Check with `in` first, or use `scores.get("Lina", 0)` to get a default.
- ▶ Square brackets do both jobs: `d[key]` reads, `d[key] = value` writes.

```
scores = {"Sara": 92, "Ali": 78, "Omar": 85}
for name, score in scores.items():      # both key and value
    print(f"{name}: {score}")
# Sara: 92
# Ali: 78
# Omar: 85
for name in scores:                     # keys only
    print(name)
```

`scores.keys()` gives the keys, `scores.values()` the values, `scores.items()` the pairs.
Pairs come out in the order they were added.

How many of each label are in a list? Use a dictionary as a set of counters. `labels = ["cat", "dog", "cat", "cat", "dog"]`

Step	Label seen	counts after
start	–	<code>{}</code>
1	cat	<code>{"cat": 1}</code>
2	dog	<code>{"cat": 1, "dog": 1}</code>
3	cat	<code>{"cat": 2, "dog": 1}</code>
4	cat	<code>{"cat": 3, "dog": 1}</code>
5	dog	<code>{"cat": 3, "dog": 2}</code>

A new label gets a counter starting at 1; a known label adds 1 to its counter. On Monday, pandas does this in one call (`value_counts`). Today you write it yourself.

```
labels = ["cat", "dog", "cat", "cat", "dog"]
counts = {}
for label in labels:
    if label in counts:
        counts[label] = counts[label] + 1    # known label: add one
    else:
        counts[label] = 1                    # new label: start at one
print(counts)    # {'cat': 3, 'dog': 2}
```

Every Olympiad task starts with this question: how many examples of each class do we have?
In Lab 2 you count 150 flowers.

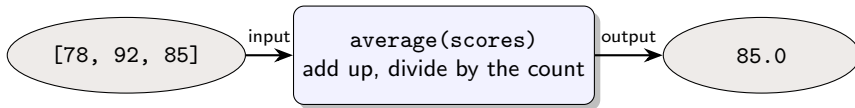
```
rows = [  
    {"name": "Sara", "score": 92, "passed": True},  
    {"name": "Ali", "score": 78, "passed": True},  
    {"name": "Lina", "score": 44, "passed": False},  
]  
rows[0]["name"]          # "Sara"      row 0, column "name"  
for row in rows:  
    print(row["name"], row["score"])
```

- ▶ Each dictionary is one row; the keys are the column names.
- ▶ This is exactly what Lab 2 loads from a CSV file, one dictionary per flower.

Python Basics: **Functions**

Definition

A **function** is a block of code with a name. It takes **inputs** (arguments), does its steps, and gives back an **output** (the return value). Write it once, call it as often as you like.



- ▶ You already use functions: `print`, `len`, `int`, `sum`. Today you write your own.
- ▶ **Not to be confused with:** a method (`name.upper()`) is a function that belongs to a value. More on that with classes.

```
def average(scores):  
    total = 0  
    for s in scores:  
        total = total + s  
    return total / len(scores)  
  
result = average([78, 92, 85])  
print(result)           # 85.0
```

1. Line 7 **calls** the function with the list [78, 92, 85].
2. Python jumps to line 1. Inside, the box scores holds that list.
3. Lines 2 to 4 run: total becomes 255.
4. Line 5 **returns** $255/3 = 85.0$ and jumps back to line 7.
5. The returned value lands in result.

def defines, the name in brackets is the **parameter**, return sends the answer back.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- ▶ x_i : the i -th score n : number of scores (`len(scores)`) $\bar{x}$: the mean
- ▶ **Worked example:** $n = 5$, scores 78, 92, 85, 64, 91
- ▶ $\sum x_i = 78 + 92 + 85 + 64 + 91 = 410$, so $\bar{x} = \frac{1}{5} \cdot 410 = 82.0$
- ▶ In the function: the loop computes $\sum x_i$, the `return` line divides by n .

Why it matters

The mean is the first number you compute about any column of any dataset. Tomorrow it becomes one NumPy call; today you own the formula.

```
def grade(score, pass_mark=50):           # pass_mark is optional
    if score >= pass_mark:
        return "pass"
    return "fail"
```

```
grade(60)                                # "pass"    uses pass_mark = 50
grade(60, pass_mark=70)                   # "fail"
```

```
def min_max(scores):
    return min(scores), max(scores)       # two values, as a tuple
lo, hi = min_max([78, 92, 85])           # lo = 78, hi = 92
```

A return ends the function at once: in grade, the last line runs only if the if was False.

Three classes, one grading rule. The rule changes: the pass mark moves from 50 to 60.

	Copy the code three times	One function, three calls
Lines to write	$3 \times 8 = 24$	$8 + 3 = 11$
Places to change the rule	3 (miss one, and one class is graded wrong)	1
Places a typo can hide	3 copies	1
Reading it a week later	which copy is the right one?	the name says what it does

The rule

If you paste the same lines twice, make them a function. Every notebook you see from now on is a list of functions and calls.

```
def average(scores):  
    total = 0  
    for s in scores:  
        total = total + s  
    return total / len(scores)
```

```
average([78, 92, 85])  
print(total)          # NameError: name 'total' is not defined
```

Local variables

`total` and `scores` exist only while the function runs. They are **local**. To use a result outside, return it and store it: `result = average(...)`.

Python Basics: **Reading Errors**

Three Errors You Will Meet Today

Error	Example line	What Python means	The fix
NameError	<code>print(scor)</code>	I do not know a name scor	typo, or its cell did not run yet
TypeError	<code>"85" + 5</code>	these two types do not mix	convert: <code>int("85") + 5</code>
IndexError	<code>scores[5]</code>	no item at that position	the last index is <code>len(scores) - 1</code>
KeyError	<code>d["Lina"]</code>	no such key in the dictionary	check with <code>in</code> ; spell the key right
ValueError	<code>int("3.7")</code>	right type, wrong content	<code>float("3.7")</code>

- ▶ The error name tells you the *kind* of problem. The message after the colon tells you the *detail*.
- ▶ An `IndentationError` or `SyntaxError` means Python could not even read the line: look at spaces, colons and brackets.

1. **Read the last line** of the error: the kind and the detail.
2. **Find the line number** it points to.
3. **Print the variables** used on that line, just before it.
4. **Check their types** with `type()`.
5. **Fix one thing**, rerun the cell, read again.

```
score = "85"  
print(score, type(score))    # 85 <class 'str'> text!  
bonus = score + 10           # TypeError: str + int
```

The print on line 2 shows the cause before line 3 fails. Printing is the beginner's best debugging tool, and the expert's too.

Python Basics: **Classes and Objects**

A quiz game with players. Each player has a name and a score. Points get added during the game.

```
sara = {"name": "Sara", "score": 0}
ali  = {"name": "Ali",   "score": 0}
omar = {"name": "Omar",  "score": 0}

def add_points(player, n):
    player["score"] = player["score"] + n

add_points(sara, 5)
print(sara)          # {'name': 'Sara', 'score': 5}
```

Three dictionaries and one function. It works. The data (the dictionaries) and the code that acts on it (the function) live apart.

The game grows: players now have a team. We add the key to two players and forget the third.

```
sara["team"] = "red"
ali["team"] = "blue"

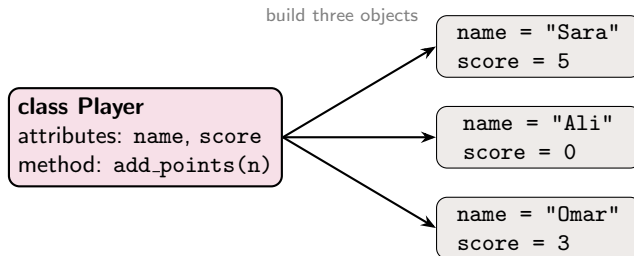
for player in [sara, ali, omar]:
    print(player["team"])      # red, blue, then KeyError: 'team'
```

The problem

Nothing forces the three dictionaries to have the same keys, and nothing ties `add_points` to the players it is meant for. With 10 functions and 100 players, one misspelled key somewhere breaks the whole run, and the error appears far from the mistake.

Definition

A **class** is a blueprint: it says what data a thing has (**attributes**) and what it can do (**methods**). An **object** is one thing built from the blueprint. Every object has the same attributes and methods, with its own values.



One blueprint, three objects. Add team to the blueprint once, and every object has it.

```
class Player:
    def __init__(self, name):
        self.name = name
        self.score = 0

    def add_points(self, n):
        self.score = self.score + n
```

- ▶ `class Player:` starts the blueprint. Class names start with a capital letter.
- ▶ `__init__` runs automatically when an object is built. It sets up the attributes.
- ▶ `self` means “this object”. `self.score` is *this* player’s score.
- ▶ A method is a function written inside the class. Its first parameter is always `self`.

```
p = Player("Sara")
```

Step	What Python does	The object
1	makes a new, empty Player object	{}
2	calls <code>__init__</code> with <code>self = that object</code> , <code>name = "Sara"</code>	{}
3	runs <code>self.name = name</code>	{name: "Sara"}
4	runs <code>self.score = 0</code>	{name: "Sara", score: 0}
5	stores the object in the box <code>p</code>	

You never call `__init__` yourself and you never pass `self`. Python fills it in. `Player("Sara")` is all you write.

```
p = Player("Sara")
p.add_points(5)           # Python passes p as self
print(p.name, p.score)    # Sara 5

players = [Player("Sara"), Player("Ali"), Player("Omar")]
players[1].add_points(3)
for player in players:
    print(player.name, player.score)    # Sara 0 / Ali 3 / Omar 0
```

- ▶ `object.attribute` reads data; `object.method(...)` does an action. That dot is the whole syntax.
- ▶ A list of objects is a list like any other: loop over it, index it, count it.

Every tool in this course is a class. You will mostly *use* objects, not write classes.

You write	What it is	When
<code>df = pd.read_csv("data.csv"); df.head()</code>	a DataFrame object and its method	Monday
<code>model = LinearRegression(); model.fit(X, y)</code>	a model object; <code>fit</code> is a method	Week 2
<code>class Net(nn.Module) with forward</code>	your own class built on PyTorch's	Week 3

Skipped on purpose today

Inheritance (a class built on another class), special methods like `__len__`, and when to write a class at all. Day 04 (self-study) covers them, with the Dataset pattern you need in Week 3.

Python Basics: **Importing Modules**

A **module** is a file of ready-made functions. `import` loads it; the dot reaches inside.

```
import math
math.sqrt(16)           # 4.0
math.pi                 # 3.141592653589793

import random
random.seed(0)           # same "random" numbers every run
random.randint(1, 6)     # a whole number from 1 to 6

from random import randint
randint(1, 6)           # the same function, imported by name
```

- ▶ `module.function(...)`: same dot as a method, because a module is a box of functions.
- ▶ `random.seed(0)` matters in every lab: it makes your run and the solution's run agree.

Tomorrow: import numpy as np

```
import numpy as np                    # "as np": a short name
scores = np.array([78, 92, 85, 64, 91])
scores.mean()                        # 82.0  today's loop in one call
```

- ▶ Nothing to learn now. Tomorrow's deck introduces arrays: the loops you wrote today become one line, and run much faster.
- ▶ Monday adds `import pandas as pd` for tables and `import matplotlib.pyplot as plt` for plots.

Python Basics: **Olympiad Practice**

Task

You are given three lists of the same length: row ids, the true labels, and the labels your program predicted.

1. Compute the **accuracy**: the number of correct predictions divided by the total number of predictions.
 2. Write the predictions to `submission.csv`: a header line `id,label`, then one line per row.
- Scoring: accuracy on the hidden test set. A file with the wrong number of lines scores 0.

```
ids      = [1, 2, 3, 4, 5]
y_true   = ["cat", "dog", "cat", "cat", "dog"]
y_pred   = ["cat", "dog", "dog", "cat", "dog"]
```

Adapted from: Deep-ML Open Problem Bank, problem 36 “Calculate Accuracy Score”, and the Kaggle submission format used in IOAI-style contests.

1. **Recognise:** “how many are equal” is the counting pattern over two lists walked together: `zip` + a counter + a division. “One line per row” is a loop over `zip(ids, y_pred)` with an f-string.
2. **Plan:**
 - `correct = 0`; for each pair `(t, p)`: if `t == p`, add 1.
 - `accuracy = correct / len(y_true)`.
 - Open the file for writing, write the header, then write `f"{i},{p}\n"` for every pair.
3. **Check:** accuracy must be between 0 and 1. Here 4 of 5 match, so $4/5 = 0.8$. The file must have $5 + 1 = 6$ lines. Open it and look before you submit.

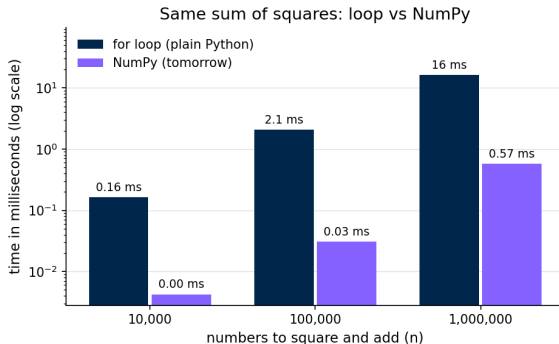
```
correct = 0
for t, p in zip(y_true, y_pred):
    if t == p:
        correct = correct + 1
accuracy = correct / len(y_true)
print(f"accuracy = {accuracy:.2f}")           # accuracy = 0.80

with open("submission.csv", "w") as f:      # "w" = write
    f.write("id,label\n")                  # \n ends the line
    for i, p in zip(ids, y_pred):
        f.write(f"{i},{p}\n")
```

`with open(...)` as `f` opens the file and closes it for you at the end of the block. This is the last cell of every Friday notebook.

Python Basics: **Limitations & What Comes Next**

Where Plain Python Gets Slow



Source: scripts/day01_loop-vs-numpy.py, measured on the instructor's laptop; sum of squares of n numbers.

- ▶ A for loop handles one number at a time. For a million numbers that is tens of milliseconds; for a million rows with many columns, minutes.
- ▶ NumPy (tomorrow) does the same arithmetic on the whole list at once, about 25 to 50 times faster here, and far more on bigger jobs.
- ▶ Today's loops are for understanding. The libraries are for speed. You need both.

- ▶ **Typing data by hand does not scale.** Real tables come from files; Lab 2 reads a CSV of 150 rows, and Monday's pandas reads 30,000 as easily.
- ▶ **Lists and dictionaries have no shape.** Nothing stops a row from having 3 values and the next 4. Tomorrow's arrays fix this.

The three mistakes to watch for

- ▶ **Off by one:** `range(5)` ends at 4; the last index is `len - 1`; a slice stops *before* its end.
- ▶ **Indentation:** a line in the wrong block runs at the wrong time, often without any error.
- ▶ **= versus ==:** storing when you meant to compare.

- ▶ **Tomorrow, Day 02: Math Foundations, Vectors & Matrices.** A list of numbers becomes a vector, a list of lists becomes a matrix, and NumPy makes today's loops one line.
- ▶ **Monday, Day 03: EDA, Pandas, Visualization.** The list of dictionaries becomes a DataFrame; counting and averaging become one call; and we draw the data.
- ▶ **Tuesday, Day 04: Python OOP (self-study).** Classes in depth: inheritance and the Dataset pattern.
- ▶ **Friday:** the Coursera exam and a lab exercise that feels like today's Lab 2 without the scaffolding.

Python Basics: **Summary**

- ▶ A **program** is a list of instructions; a **variable** is a named box; a notebook runs it cell by cell.
- ▶ Values have **types**: `int`, `float`, `str`, `bool`. **Lists** hold values by position, **dictionaries** by key.
- ▶ `if/elif/else` chooses; `for` repeats over items; `while` repeats until a condition fails. The **accumulator** and **counting** patterns do most of the work.
- ▶ A **function** names a recipe: `def`, parameters, `return`. Write it once, call it many times.
- ▶ A **class** is a blueprint, an **object** is one thing built from it; `__init__` sets it up and `self` is this object.
- ▶ An **error** names its kind and its line. Read the last line, print the variables, fix one thing.

Python Basics: **References & Resources**

- [1] Python Software Foundation (2026). *The Python Tutorial*.
<https://docs.python.org/3/tutorial/>
- [2] Open-Deep-ML (2025). *DML-OpenProblem*, problem 36: Calculate Accuracy Score. The practice task is adapted from it.
<https://github.com/Open-Deep-ML/DML-OpenProblem>
- [3] Becker, B. and Kohavi, R. (1996). *Adult* data set (32,561 training rows). UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/2/adult>
- [4] Fisher, R. A. (1936). *Iris* data set, Kaggle copy uciml/iris, used in Lab 2.
<https://www.kaggle.com/datasets/uciml/iris>
- [5] Google (2026). *Welcome to Colab*. <https://colab.research.google.com/>

- [6] Altwaim, S. (2026). *Day 1: Math & Code Foundations*, KAUST Academy & Mawhiba IOAI 2026 training. Grade-letter, average and class examples adapted from it.